ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Marcin Migacz

Klasa: 5P

Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026r.

Spis treści

Wstęp 4
1. Testy jednostkowe 5
   1.1. Implementacja programu 5
   1.2. Implementacja testów 9
   1.3. Git 11
2. Testy integracyjne 12
   2.1. Implementacja programu 12
   2.2. Implementacja testów 12
   2.3. GIT 15
3. Testy systemowe 16
   3.1. Implementacja programu 16
   3.2. Implementacja testów 16
   3.3. GIT 18
4. Testy akceptacyjne 19
   4.1. Implementacja programu 19
   4.2. Implementacja testów 19
   4.3. GIT 21
Podsumowanie 22
Bibliografia 22

Wstęp

Testowanie oprogramowania to proces sprawdzania, czy program działa zgodnie z wymaganiami oraz wykrywania w nim błędów przed przekazaniem go użytkownikom. Błąd (defekt) to wada w kodzie, która może doprowadzić do awarii, czyli widocznego niepoprawnego działania systemu.

Celem testowania jest nie tylko znajdowanie usterek, ale też ograniczanie ryzyka jego wdrożenia. Ze względu na poziom wyróżnia się testy jednostkowe, integracyjne, systemowe i akceptacyjne, które sprawdzają kolejno pojedyncze elementy, ich współpracę, cały system oraz zgodność z oczekiwaniami klienta. Ze względu na cel dzielimy je na funkcjonalne (co system robi) i niefunkcjonalne (jak działa: wydajność, bezpieczeństwo, użyteczność), a osobną grupę stanowią testy regresyjne, które po zmianach w kodzie sprawdzają, czy nie zepsuto istniejących funkcji. Wyróżnia się też podejścia czarnej skrzynki (bez znajomości kodu), białej skrzynki (z analizą wnętrza programu) i szarej skrzynki (mieszane).

Pierwsza zasada testowania mówi, że testy ujawniają obecność błędów, ale nie dowodzą ich braku. Druga głosi, że testowanie wyczerpujące jest niemożliwe, więc wysiłek kieruje się na obszary o największym ryzyku. Trzecia zasada to wczesne testowanie, bo błąd znaleziony na etapie projektu kosztuje znacznie mniej niż ten znaleziony po wdrożeniu. Kolejne zasady mówią, że defekty grupują się w niewielu modułach, a wielokrotne uruchamianie tych samych testów przestaje wykrywać nowe błędy. Ostatnie zasady to zależność testów od kontekstu oraz złudzenie braku błędów: program bez usterek nadal nie jest wartościowy, jeśli nie spełnia potrzeb użytkownika.

Proces testowy zaczyna się od planowania i nadzoru, czyli określenia zakresu, zasobów i kryteriów zakończenia. Następnie następuje analiza i projektowanie testów, w których powstają przypadki testowe z danymi wejściowymi i oczekiwanymi wynikami. Potem testy są implementowane i wykonywane, a wyniki rzeczywiste porównuje się z oczekiwanymi, po czym raportuje się znalezione błędy, wykonuje retesty po poprawkach i zamyka proces podsumowaniem.

1. Testy jednostkowe

Czym są? - Testy jednostkowe (ang. *unit tests*) sprawdzają najmniejsze samodzielne fragmenty programu, np. pojedynczą metodę lub klasę, w izolacji od reszty systemu. Każdy test podaje określone dane wejściowe, uruchamia testowaną jednostkę i porównuje jej wynik z oczekiwanym. Są szybkie, w pełni automatyczne i powtarzalne, więc można je uruchamiać po każdej zmianie kodu.

Kiedy je wykonujemy? - Najczęściej w trakcie pisania kodu, zwykle przez samego programistę, a potem po każdej modyfikacji oraz automatycznie w procesie CI/CD przed scaleniem zmian. Dzięki temu błąd jest wykrywany w miejscu, w którym powstał i jest tani w naprawie.

Narzędzie - xUnit. xUnit to popularny, darmowy framework do testów jednostkowych w .NET (C#). Testy są zwykłymi metodami oznaczonymi atrybutem [Fact] (test bez parametrów) lub [Theory] (test z wieloma zestawami danych). Do sprawdzania wyników służą asercje klasy Assert. Test przechodzi, gdy wszystkie asercje są spełnione, a nie przechodzi, gdy któraś zawiedzie lub wystąpi nieoczekiwany wyjątek.

1.1. Implementacja programu

Program to konsolowa loteria liczbowa typu „Lotto”. Użytkownik podaje, ile zestawów chce wylosować. Program losuje dla każdego zestawu 6 różnych liczb z przedziału 1–49 i zapisuje je w dwuwymiarowej tablicy (wiersz = zestaw, kolumna = pozycja w zestawie). Następnie wyświetla wszystkie zestawy i zlicza, ile razy w całym losowaniu wystąpiła każda liczba od 1 do 49. Gdy użytkownik wpisze tekst zamiast liczby, program wypisuje komunikat o błędzie i kończy działanie.

Przykładowy wynik dla wejścia 2:

```
Ile wygenerować losowań?
2
Zestawy wylosowanych liczb:
Losowanie 1: 7 23 4 41 19 35
Losowanie 2: 12 3 48 27 7 15
Wystąpienia liczby 1: 0
Wystąpienia liczby 2: 0
Wystąpienia liczby 3: 1
(Wypisywane wystąpienia kolejnych liczb aż do 49)
```

Opis kodu:

- **Stałe:**

LiczbWZestawie	6	Ile liczb zawiera jeden zestaw (jedno losowanie).
MinLiczba	1	Najmniejsza liczba, którą można wylosować.
MaxLiczba	49	Największa liczba, którą można wylosować

• **Funkcja Main**

1. **int.TryParse(Console.ReadLine(), out int ileZestawow)** - wczytuje tekst z klawiatury i próbuje zamienić go na liczbę całkowitą. Jeśli się uda, zwraca true, a wynik trafia do zmiennej ileZestawow. Jeśli użytkownik wpisze tekst (np. abc), zwraca false i program nie ulega awarii.
2. **if (poprawne dane):**
   - **new int[ileZestawow, LiczbWZestawie]** tworzy dwuwymiarową tablicę losowania o n wierszach (zestawy) i 6 kolumnach (liczby w zestawie).
   - Pętla zewnętrzna for (i...) wykonuje się raz dla każdego zestawu. W każdym obiegu wywołuje LosujZestaw(), która zwraca 6-elementową tablicę.
   - Pętla wewnętrzna for (j...) przepisuje 6 wylosowanych liczb do wiersza i tablicy losowania.
   - **WyswietlLosowania(ileZestawow, losowania)** wypisuje wyniki i statystykę.
3. **else (błędne dane):** wypisuje komunikat o niepoprawnej wartości i kończy program instrukcją return.

• **Funkcja LosujZestaw()** - losuje jeden zestaw 6 różnych liczb z przedziału 1–49. Nie przyjmuje argumentów i zwraca tablicę int[] z wylosowanymi liczbami.

1. **int[] zestaw = new int[LiczbWZestawie]** - tworzy pustą tablicę na 6 liczb. Początkowo wszystkie elementy mają wartość 0.
2. **Random generator = new Random()** - tworzy generator liczb pseudolosowych.
3. **int i = 0** to licznik zapisanych już liczb, a zarazem indeks następnego wolnego miejsca w zestawie.
4. Pętla **while (i < LiczbWZestawie)** działa, dopóki zestaw nie zawiera 6 liczb.
5. **generator.Next(MinLiczba, MaxLiczba + 1)** losuje liczbę całkowitą. Górna granica w Next wynosi o 1 mniej niż dana liczba, dlatego dodano + 1, żeby 49 mogło zostać wylosowane.
6. **zestaw.Contains(wylosowana)** sprawdza, czy taka liczba już jest w zestawie:

   jeśli tak, **continue** przerywa obieg pętli i losowanie zaczyna się od nowa, a licznik i się nie zwiększa,

   jeśli nie, liczba zostaje zapisana pod indeksem i, a następnie i++ przesuwa licznik.

7. **return** zestaw zwraca gotowy zestaw.

• **Funkcja WyswietlLosowania()** – losuje

1. **int[] wystapienia = new int[MaxLiczba]** - tworzy tablicę 49 liczników. Indeks liczba - 1 odpowiada liczbie z loterii, czyli liczba 1 jest pod indeksem 0, a liczba 49 pod indeksem 48.
2. Pierwsza pętla for zeruje wszystkie liczniki.
3. Zagnieżdżone pętle for przechodzą po wszystkich zestawach (i) i liczbach w zestawie (j):
   - **Console.Write($"Losowanie {i + 1}: ")** - wypisuje numer losowania, numerowany od 1 dla użytkownika,
   - **wystapienia[losowania[i, j] - 1]++** - zwiększa licznik wylosowanej liczby o 1,
   - **Console.Write(losowania[i, j] + " ")** - wypisuje liczbę, a po niej spację,
   - **Console.WriteLine("")** - - po 6 liczbach przechodzi do nowej linii.
4. Ostatnia pętla for wypisuje dla każdej liczby od 1 do 49 tekst „Wystąpienia liczby X: Y”, gdzie Y to wartość licznika.

```
using System;

namespace Loteria
{
    Odwołania: 4
    public class Program
    {
        Odwołania: 5
        const int LiczbWZestawie = 6;
        1 odwołanie
        const int MinLiczba = 1;
        Odwołania: 2
        const int MaxLiczba = 49;

        Odwołania: 0
        public static void Main(string[] args)
        {
            Console.WriteLine("Ile wygenerować losowań?");

            if (int.TryParse(Console.ReadLine(), out int ileZestawow))
            {
                int[,] losowania = new int[ileZestawow, LiczbWZestawie];

                for (int i = 0; i < ileZestawow; i++)
                {
                    int[] zestaw = LosujZestaw();
                    for (int j = 0; j < LiczbWZestawie; j++)
                    {
                        losowania[i, j] = zestaw[j];
                    }
                }

                WyswietlLosowania(ileZestawow, losowania);
            }
            else
            {
                Console.WriteLine("Niepoprawna wartość. Podaj liczbę całkowitą z zakresu 1-49");
                return;
            }
        }

        Odwołania: 4 | 3/3 passing
        public static int[] LosujZestaw()
        {
            int[] zestaw = new int[LiczbWZestawie];
            Random generator = new Random();
            int i = 0;

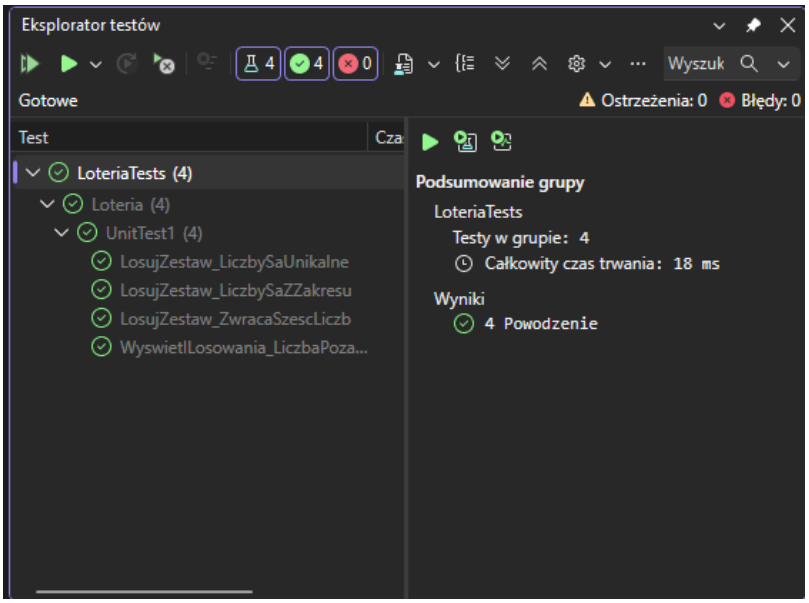
            while (i < LiczbWZestawie)
            {
                int wylosowana = generator.Next(MinLiczba, MaxLiczba + 1);
                if (zestaw.Contains(wylosowana))
                {
                    continue;
                }
                else
                {
                    zestaw[i] = wylosowana;
                }

                i++;
            }
            return zestaw;
        }
```

```
59
                Odwołania: 2 | ● 1/1 passing
60          public static void WyswietlLosowania(int liczbaZestawow, int[,] losowania)
61          {
62              Console.WriteLine("Zestawy wylosowanych liczb:");
63              //[indeks + 1 -> liczba, jej wystąpienia]
64              int[] wystapienia = new int[MaxLiczba];
65
66              for(int i = 0; i < wystapienia.Length; i++)
67              {
68                  wystapienia[i] = 0;
69              }
70
71              for (int i = 0; i < liczbaZestawow; i++)
72              {
73                  Console.Write($"Losowanie {i + 1}: ");
74
75                  for (int j = 0; j < LiczbWZestawie; j++)
76                  {
77                      //Zwiększenie liczby wystąpień danej liczby
78                      wystapienia[losowania[i, j] - 1]++;
79
80                      Console.Write(losowania[i, j] + " ");
81                  }
82
83                  Console.WriteLine("");
84              }
85
86              for (int i = 0; i < wystapienia.Length; i++)
87              {
88                  Console.WriteLine($"Wystąpienia liczby {i + 1}: {wystapienia[i]}");
89              }
90
91          }
92      }
93  }
94
```

1.2. Implementacja testów

#	Typ	Metoda testowa	Opis	Oczekiwany wynik
1	poprawny	LosujZestaw_ZwracaSzescLiczb	Wywołuje LosujZestaw() i sprawdza Assert.Equal(6, zestaw.Length).	Zestaw ma dokładnie 6 elementów. Test zdany.
2	poprawny	LosujZestaw_LiczbySaUnikalne	Losuje 1000 zestawów. Distinct() usuwa powtórzenia, więc liczba elementów po jego zastosowaniu musi być równa długości zestawu.	W żadnym zestawie nie ma duplikatów. Test zdany.
3	poprawny	LosujZestaw_LiczbySaZZakresu	Losuje 1000 zestawów i dla każdej liczby sprawdza Assert.True(liczba >= 1 && liczba <= 49).	Każda liczba mieści się w przedziale 1–49. Test zdany.
4	niepoprawny	WyswietlLosowania_LiczbaPoza_Zakresem_RzucaWyjatek	Dane zawierają liczbę 50, spoza zakresu. Metoda próbuje zwiększyć wystapienia[49], ale tablica ma indeksy 0–48. Assert.Throws sprawdza, że rzucany jest IndexOutOfRangeException.	Wyjątek zostaje rzucony, więc test jest zdany.



LoteriaTests › Loteria.UnitTest1

```
namespace Loteria
{
    Odwołania: 0
    public class UnitTest1
    {
        // TEST 1 (poprawny): zestaw zawiera dokładnie 6 liczb
        [Fact]
        Odwołania: 0
        public void LosujZestaw_ZwracaSzescLiczb()
        {
            int[] zestaw = Program.LosujZestaw();

            Assert.Equal(6, zestaw.Length);
        }

        // TEST 2 (poprawny): liczby w zestawie są unikalne
        [Fact]
        Odwołania: 0
        public void LosujZestaw_LiczbySaUnikalne()
        {
            //Przypadek powtarzany wiele razy, by ominąć element losowości
            for (int powtorzenie = 0; powtorzenie < 1000; powtorzenie++)
            {
                int[] zestaw = Program.LosujZestaw();

                Assert.Equal(zestaw.Length, zestaw.Distinct().Count());
            }
        }

        // TEST 3 (poprawny): liczby w zestawie są z zakresu 1–49
        [Fact]
        Odwołania: 0
        public void LosujZestaw_LiczbySaZZakresu()
        {
            for (int powtorzenie = 0; powtorzenie < 1000; powtorzenie++)
            {
                int[] zestaw = Program.LosujZestaw();

                foreach (int liczba in zestaw)
                {
                    Assert.True(liczba >= 1 && liczba <= 49);
                }
            }
        }

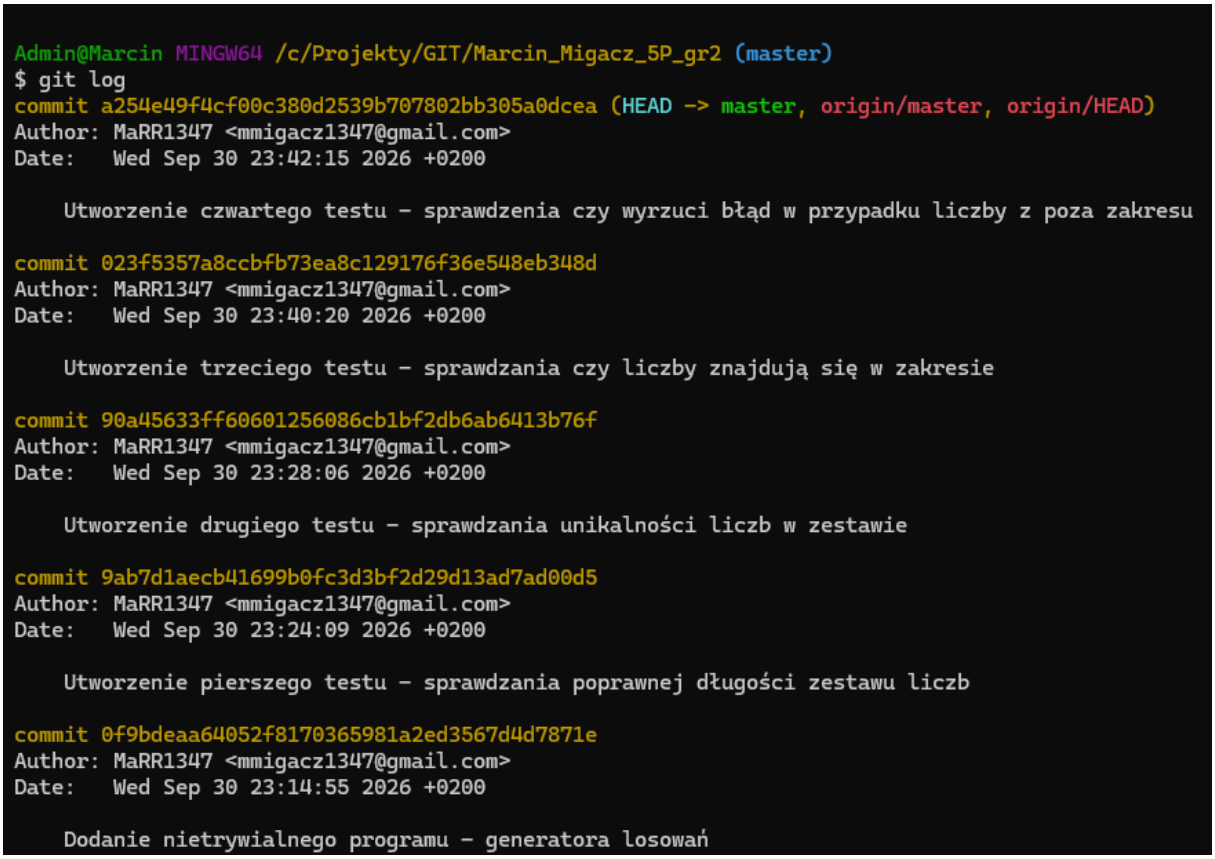
        // TEST 4 (niepoprawny): liczba 50 jest poza zakresem
        [Fact]
        Odwołania: 0
        public void WyswietlLosowania_LiczbaPoza_Zakresem_RzucaWyjatek()
        {
            int[,] losowania = { { 1, 2, 3, 4, 5, 50 } };

            Assert.Throws<IndexOutOfRangeException>(() => Program.WyswietlLosowania(1, losowania));
        }

    }
}
```

1.3. Git

Screen z git log commitów wprowadzających przypadki testowe:



```
Admin@Marcin MINGW64 /c/Projekty/GIT/Marcin_Migacz_5P_gr2 (master)
$ git log
commit a254e49f4cf00c380d2539b707802bb305a0dcea (HEAD -> master, origin/master, origin/HEAD)
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:42:15 2026 +0200

    Utworzenie czwartego testu - sprawdzenia czy wyrzuci błąd w przypadku liczby z poza zakresu

commit 023f5357a8ccbfb73ea8c129176f36e548eb348d
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:40:20 2026 +0200

    Utworzenie trzeciego testu - sprawdzania czy liczby znajdują się w zakresie

commit 90a45633ff60601256086cb1bf2db6ab6413b76f
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:28:06 2026 +0200

    Utworzenie drugiego testu - sprawdzania unikalności liczb w zestawie

commit 9ab7d1aecb41699b0fc3d3bf2d29d13ad7ad00d5
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:24:09 2026 +0200

    Utworzenie pierszego testu - sprawdzania poprawnej długości zestawu liczb

commit 0f9bdeaa64052f8170365981a2ed3567d4d7871e
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:14:55 2026 +0200

    Dodanie nietrywialnego programu - generatora losowań
```

2. Testy integracyjne

Czym są? – Testy integracyjne (ang. *integration tests*) sprawdzają, czy połączone moduły i komponenty aplikacji współpracują ze sobą prawidłowo. W przeciwieństwie do testów jednostkowych nie izolują kodu, lecz weryfikują jego interakcje z prawdziwymi elementami zewnętrznymi, takimi jak baza danych, pliki systemowe, zewnętrzne interfejsy API czy kolejki komunikatów.

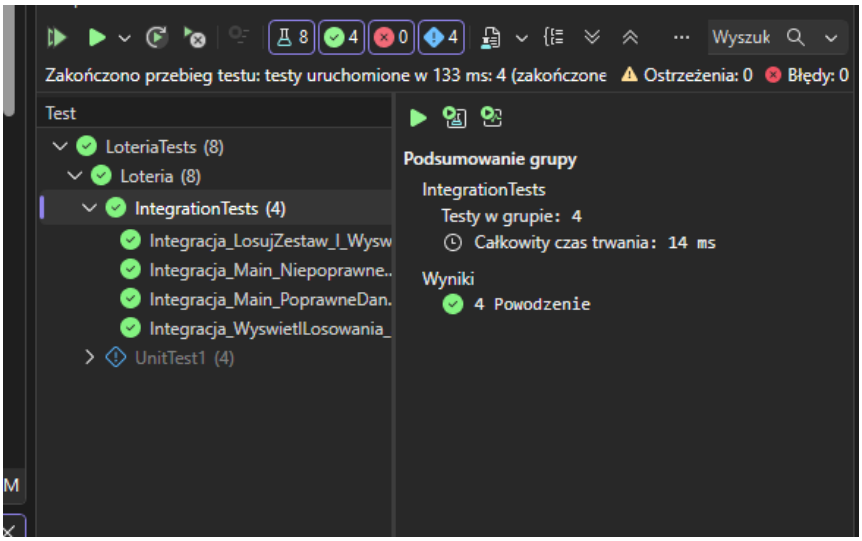
Kiedy je wykonujemy? – Wykonuje się je po przejściu testów jednostkowych, gdy poszczególne moduły są już gotowe i połączone ze sobą. Są również uruchamiane automatycznie w potokach CI/CD (np. na osobnym środowisku testowym), aby upewnić się, że zmiana w jednym module nie zepsuła komunikacji z resztą systemu.

2.1. Implementacja programu

Ten rozdział będzie się opierać na programie opisanym w poprzednim punkcie. Szczegóły jego implementacji znajdują się w rozdziale 1.1. dokumentacji.

2.2. Implementacja testów

#	Typ	Metoda testowa	Opis	Oczekiwany wynik
1	poprawny	Integracja_LosujZestaw_I_WyswietlLosowania_PrzetwarzaDaneBezBledow	Wywołuje LosujZestaw() wielokrotnie, przekazuje uzyskane dane do WyswietlLosowania() i przechwytuje wyjście konsoli za pomocą StringWriter.	Metoda WyswietlLosowania wykonuje się poprawnie, a na konsoli pojawiają się nagłówki losowań oraz statystyki wystąpień. Test zdany.
2	poprawny	Integracja_Main_PoprawneDane_WypisujeZestawyIStatystyki	Symuluje wpisanie w konsoli poprawnej liczby zestawów (np. "2"), przekierowując Console.In i Console.Out.	Program przetwarza wejście, generuje podaną liczbę losowań i wypisuje pełne podsumowanie w konsoli. Test zdany.
3	poprawny	Integracja_WyswietlLosowania_PoprawnieZliczaSumeWystapienLiczb	Generuje znaną liczbę zestawów (np. 5) i weryfikuje w wypisanym strumieniu wyjściowym, czy łączna suma wystąpień wszystkich liczb wynosi dokładnie $5 \times 6 = 30$.	Suma zliczonych wystąpień zgadza się z łączną liczbą wylosowanych elementów. Test zdany.
4	niepoprawny	Integracja_Main_NiepoprawneDane_WypisujeKomunikatOBledzie	Symuluje wpisanie błędnych danych (np. "abc" lub "-5") poprzez przekierowanie Console.In.	Program wypisuje komunikat o niepoprawnej wartości, nie wywołuje losowania i bezpiecznie kończy działanie. Test zdany.



UnitTest2.cs | UnitTest1.cs | Program.cs

LoteriaTests | Loteria.IntegrationTests

```
using System.Text.RegularExpressions;

namespace Loteria
{
    Odwołania: 0
    public class IntegrationTests
    {
        // TEST 1 (poprawny): Integracja modułu losującego z modułem wyświetlającym
        [Fact]
        Odwołania: 0
        public void Integracja_LosujZestaw_I_WyswietlLosowania_PrzetwarzaDaneBezBledow()
        {
            int ileZestawow = 3;
            int[,] losowania = new int[ileZestawow, 6];

            for (int i = 0; i < ileZestawow; i++)
            {
                int[] zestaw = Program.LosujZestaw();
                for (int j = 0; j < 6; j++)
                {
                    losowania[i, j] = zestaw[j];
                }
            }

            using (StringWriter sw = new StringWriter())
            {
                Console.SetOut(sw);

                Program.WyswietlLosowania(ileZestawow, losowania);

                string output = sw.ToString();
                Assert.Contains("Zestawy wylosowanych liczb:", output);
                Assert.Contains("Losowanie 1:", output);
                Assert.Contains("Losowanie 2:", output);
                Assert.Contains("Losowanie 3:", output);
                Assert.Contains("Wystąpienia liczby 1:", output);
                Assert.Contains("Wystąpienia liczby 49:", output);
            }
        }

        // TEST 2 (poprawny): Integracja całej metody Main z poprawnym wejściem użytkownika
        [Fact]
        Odwołania: 0
        public void Integracja_Main_PoprawneDane_WypisujeZestawyIStatystyki()
        {
            string input = "2" + Environment.NewLine; // Użytkownik podaje 2 zestawy
            using (StringReader sr = new StringReader(input))
            using (StringWriter sw = new StringWriter())
            {
                Console.SetIn(sr);
                Console.SetOut(sw);

                Program.Main(new string[0]);

                string output = sw.ToString();
                Assert.Contains("Ile wygenerować losowań?", output);
                Assert.Contains("Zestawy wylosowanych liczb:", output);
                Assert.Contains("Losowanie 1:", output);
                Assert.Contains("Losowanie 2:", output);
                Assert.Contains("Wystąpienia liczby", output);
            }
        }
```

```
59          }
60
61          // TEST 3 (poprawny): Weryfikacja spójności danych – zliczenie łącznej sumy wystąpień w wyjściu konsoli
62          [Fact]
            Odwołania: 0
63          public void Integracja_WyswietlLosowania_PoprawnieZliczaSumeWystapienLiczb()
64          {
65              int ileZestawow = 5;
66              int[,] losowania = new int[ileZestawow, 6];
67
68              for (int i = 0; i < ileZestawow; i++)
69              {
70                  int[] zestaw = Program.LosujZestaw();
71                  for (int j = 0; j < 6; j++)
72                  {
73                      losowania[i, j] = zestaw[j];
74                  }
75              }
76
77              using (StringWriter sw = new StringWriter())
78              {
79                  Console.SetOut(sw);
80
81                  Program.WyswietlLosowania(ileZestawow, losowania);
82
83                  string output = sw.ToString();
84
85                  // Wyciągnięcie wartości po dwukropku z linii "Wystąpienia liczby X: Y"
86                  var matches = Regex.Matches(output, @"Wystąpienia liczby \d+: (\d+)");
87                  int sumaWystapien = matches.Cast<Match>().Sum(m => int.Parse(m.Groups[1].Value));
88
89                  Assert.Equal(ileZestawow * 6, sumaWystapien);
90              }
91          }
92
93          // TEST 4 (niepoprawny): Integracja metody Main z błędnym wejściem użytkownika
94          [Fact]
            Odwołania: 0
95          public void Integracja_Main_NiepoprawneDane_WypisujeKomunikatOBledzie()
96          {
97              string input = "abc" + Environment.NewLine; // Użytkownik podaje tekst zamiast liczby
98              using (StringReader sr = new StringReader(input))
99              using (StringWriter sw = new StringWriter())
100             {
101                 Console.SetIn(sr);
102                 Console.SetOut(sw);
103
104                 Program.Main(new string[0]);
105
106                 string output = sw.ToString();
107                 Assert.Contains("Niepoprawna wartość. Podaj liczbę całkowitą większą od 0", output);
108                 Assert.DoesNotContain("Zestawy wylosowanych liczb:", output);
109             }
110         }
111     }
112 }
```

2.3. GIT

Screen z git log commitów wprowadzających przypadki testowe:

```
Admin@Marcin MINGW64 /c/Projekty/GIT/Marcin_Migacz_5P_gr2 (master)
$ git log
commit 7d4e0a8f07881db2e45b96d34336fcccb1b8945a (HEAD -> master)
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Oct 14 19:16:41 2026 +0200

    Utworzenie czwartego testu integracyjnego - sprawdzania reakcji programu n
a niepoprawne dane wejściowe w konsoli

commit e1442871a9d19cb42012aef835e3e6be7f616835
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Oct 14 19:14:28 2026 +0200

    Utworzenie trzeciego testu integracyjnego - sprawdzania poprawności zlicza
nia łącznej sumy wystąpień liczb

commit 321c9fa751afe0518d164af7134a73152624f3c9
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Oct 14 19:09:49 2026 +0200

    Dodanie drugiego test integracyjnego sprawdzania pełnego przebiegu program
u z poprawną liczbą losowań

commit 42504ba5c1d211a0e0fa1afb1b89fe0979c6836c
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Oct 14 19:04:52 2026 +0200

    Dodanie pierwszego testu integracyjnego - Losowania danych i przetwarzania
 ich do wyświetlanej wiadomości
```

3. Testy systemowe

Czym są? – Testy systemowe (ang. *system tests*) sprawdzają cały, kompletnie zmontowany i zintegrowany produkt programistyczny jako jedną całość. Weryfikują, czy aplikacja spełnia wszystkie określone wymagania funkcjonalne i niefunkcjonalne (np. wydajność, bezpieczeństwo, użyteczność) w środowisku zbliżonym do produkcyjnego.

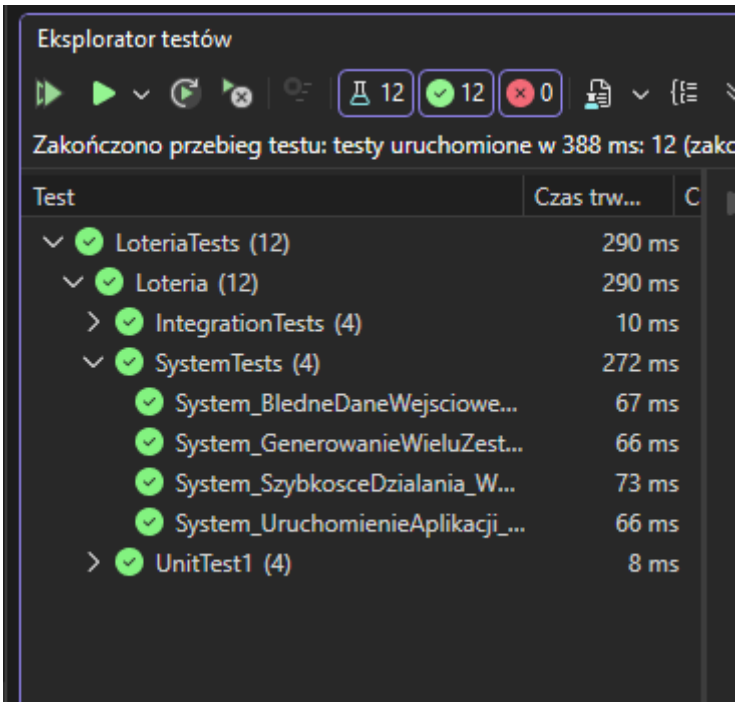
Kiedy je wykonujemy? – Wykonujemy je na późniejszym etapie tworzenia oprogramowania – po zakończeniu integracji wszystkich modułów, a przed przekazaniem aplikacji użytkownikom końcowym lub klientowi. Zwykle odpowiada za nie dedykowany zespół testerów (QA).

3.1. Implementacja programu

Podobnie jak poprzednio ten rozdział będzie się opierać na programie opisanym w pierwszym punkcie. Szczegóły jego implementacji znajdują się w rozdziale 1.1. dokumentacji.

3.2. Implementacja testów

#	Typ	Metoda testowa	Opis	Oczekiwany wynik
1	poprawny	System_UruchomienieAplikacji_ZwracaKodSukcesuZero	Uruchamia skompilowany plik wykonywalny aplikacji w osobnym procesie systemowym, przekazując na wejście poprawną liczbę losowań (np. "1").	Process kończy się kodem wyjścia 0 (*ExitCode = 0*), a w konsoli pojawia się pełny komunikat startowy i końcowy. Test zdany.
2	poprawny	System_GenerowanieWieluZestawow_FormatWyjsciaZgodnyZWymaganiami	Uruchamia proces systemowy dla argumentu wejściowego "3" i sprawdza pełną strukturę tekstu zwróconego przez system.	Wyjście systemu zawiera dokładnie 3 linie z nagłówkami Losowanie 1:, Losowanie 2:, Losowanie 3: oraz pełną listę 49 statystyk. Test zdany.
3	poprawny	System_SzybkosceDzialania_WykonanieDlaDuzejLiczbyLosowan	Uruchamia proces z dużą liczbą losowań (np. "1000") i mierzy całkowity czas wykonania procesu w systemie operacyjnym.	Cały proces uruchamia się, generuje dane i kończy działanie w czasie poniżej 2 sekund. Test zdany.
4	niepoprawny	System_BledneDaneWejsciowe_ZwracaObsluzonyBdadIObrywaProces	Uruchamia proces systemowy przekazując niepoprawny ciąg znaków (np. "nie_liczba").	Proces wypisuje w systemie komunikat o błędzie i natychmiast bezpiecznie zamyka się z kodem wyjścia 0 (brak awarii/crasha aplikacji). Test zdany.



LoteriaTests | Loteria.SystemTests

```
using System.Diagnostics;

namespace Loteria
{
    Odwołania: 0
    public class SystemTests
    {
        // Ścieżka do skompilowanego pliku wykonywalnego aplikacji (.exe)
        1 odwołanie
        private readonly string _sciezkaDoAplikacji = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Loteria.exe");

        // Helper uruchamiający proces systemowy aplikacji
        Odwołania: 4 | 4/4 passing
        private (int ExitCode, string Output) UruchomProcesSystemowy(string input)
        {
            ProcessStartInfo psi = new ProcessStartInfo
            {
                FileName = _sciezkaDoAplikacji,
                RedirectStandardInput = true,
                RedirectStandardOutput = true,
                UseShellExecute = false,
                CreateNoWindow = true
            };

            using (Process process = new Process { StartInfo = psi })
            {
                process.Start();

                using (StreamWriter sw = process.StandardInput)
                {
                    sw.WriteLine(input);
                }

                string output = process.StandardOutput.ReadToEnd();
                process.WaitForExit(5000); // Timeout 5 sekund

                return (process.ExitCode, output);
            }
        }

        // TEST 1 (poprawny): Poprawne uruchomienie i zakończenie procesu w systemie
        [Fact]
        Odwołania: 0
        public void System_UruchomienieAplikacji_ZwracaKodSukcesuZero()
        {
            var (exitCode, output) = UruchomProcesSystemowy("1");

            Assert.Equal(0, exitCode);
            Assert.Contains("Ile wygenerować losowań?", output);
            Assert.Contains("Zestawy wylosowanych liczb:", output);
        }
```

```
47          }
48
49          // TEST 2 (poprawny): Weryfikacja struktury wyjścia systemowego dla wielu zestawów
50          [Fact]
            Odwołania: 0
51          public void System_GenerowanieWieluZestawow_FormatWyjsciaZgodnyZWymaganiami()
52          {
53              var (exitCode, output) = UruchomProcesSystemowy("3");
54
55              Assert.Equal(0, exitCode);
56              Assert.Contains("Losowanie 1:", output);
57              Assert.Contains("Losowanie 2:", output);
58              Assert.Contains("Losowanie 3:", output);
59              Assert.DoesNotContain("Losowanie 4:", output);
60              Assert.Contains("Wystąpienia liczby 49:", output);
61          }
62
63          // TEST 3 (poprawny): Wydajność systemowa – generowanie dużej liczby losowań
64          [Fact]
            Odwołania: 0
65          public void System_SzybkosceDzialania_WykonanieDlaDuzejLiczbyLosowan()
66          {
67              // Arrange
68              Stopwatch stoper = Stopwatch.StartNew();
69
70              // Act
71              var (exitCode, output) = UruchomProcesSystemowy("1000");
72              stoper.Stop();
73
74              // Assert
75              Assert.Equal(0, exitCode);
76              Assert.Contains("Losowanie 1000:", output);
77              Assert.True(stoper.ElapsedMilliseconds < 2000, $"Czas wykonania wyniósł {stoper.ElapsedMilliseconds} ms, oczekiwano poniżej 2000 ms.");
78          }
79
80          // TEST 4 (niepoprawny): Reakcja systemu na niepoprawne dane wejściowe
81          [Fact]
            Odwołania: 0
82          public void System_BledneDaneWejsciowe_ZwracaObsluzonyBdadIObrywaProces()
83          {
84              // Act
85              var (exitCode, output) = UruchomProcesSystemowy("nie_liczba");
86
87              // Assert
88              Assert.Equal(0, exitCode); // Aplikacja nie może się skraszować (crash/exception)
89              Assert.Contains("Niepoprawna wartość. Podaj liczbę całkowitą większą od 0", output);
90              Assert.DoesNotContain("Zestawy wylosowanych liczb:", output);
91          }
92      }
93  }
```

3.3. GIT

Screen z git log commitów wprowadzających przypadki testowe:

```
Admin@Marcin MINGW64 /c/Projekty/GIT/Marcin_Migacz_5P_gr2 (master)
$ git log
commit 5911cbd4971e629ebae9083d9f0f4923e480e9db (HEAD -> master)
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 11:46:11 2026 +0200

    Utworzenie czwartego testu systemowego – stabilności procesu systemowego pr
zy błędnych danych wejściowych

commit 4dbead3964ca46beda1606a2cae708c88b99dfcc
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 11:33:35 2026 +0200

    Utworzenie trzeciego testu systemowego – wydajności aplikacji przy generowa
niu dużej liczby losowań

commit ed5ff48a837bfa489af1bc6fb8a5e1da3ba96b35
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 11:23:03 2026 +0200

    Utworzenie drugiego testu systemowego – sprawdzania formatu i kompletności
wyjścia systemowego dla wielu zestawów

commit ccef0af19b11bed8b5d9a79c263d6c2944abe6e6
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 11:21:31 2026 +0200

    Utworzenie pierwszego testu systemowego – sprawdzania poprawnego uruchomien
ia procesu aplikacji i kodu wyjścia
```

4. Testy akceptacyjne

Czym są? - Testy akceptacyjne (ang. *acceptance tests*, np. UAT – *User Acceptance Testing*) to ostateczny etap weryfikacji oprogramowania. Ich celem nie jest szukanie drobnych błędów w kodzie, ale potwierdzenie, że aplikacja spełnia biznesowe potrzeby użytkowników, działa zgodnie z umową i jest gotowa do wdrożenia produkcyjnego.

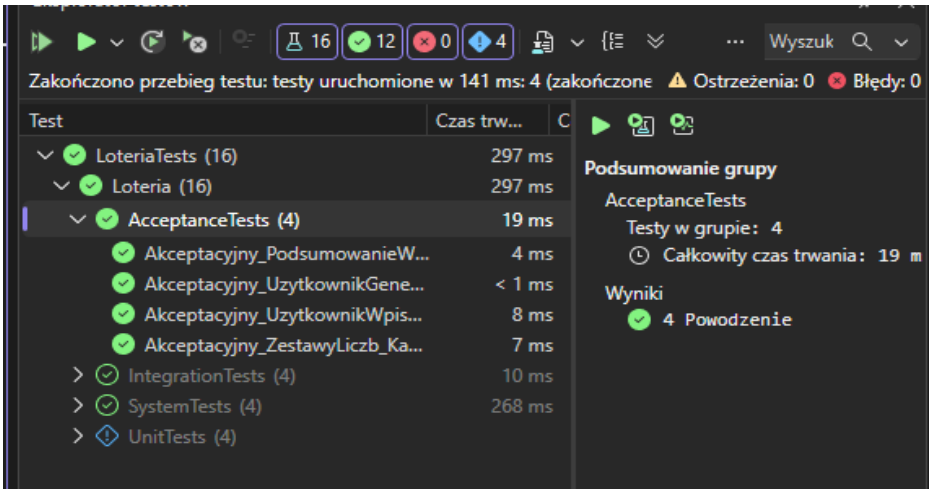
Kiedy je wykonujemy? – Wykonujemy je na samym końcu procesu wytwórczego, bezpośrednio przed wydaniem oprogramowania (lub po zakończeniu danej iteracji/sprintu). Są przeprowadzane głównie przez klienta, zamawiającego lub reprezentatywną grupę użytkowników końcowych.

4.1. Implementacja programu

Podobnie jak poprzednio, ten rozdział będzie się opierać na programie opisanym w pierwszym punkcie. Szczegóły jego implementacji znajdują się w rozdziale 1.1. dokumentacji.

4.2. Implementacja testów

#	Typ	Metoda testowa	Opis	Oczekiwany wynik
1	poprawny	AkceptacyjnyUzytkownikGenerujeZestawyOtrzymujePrawidlowaLiczbeLosowan	Symuluje scenariusz, w którym użytkownik zamawia wygenerowanie 3 zestawów losowań.	Aplikacja generuje i wyświetla dokładnie 3 zestawy losowań z nagłówkami Losowanie 1: , Losowanie 2: , Losowanie 3: . Test zdany.
2	poprawny	AkceptacyjnyZestawyLiczb_KazdyZestawRoznyIUnikalneLiczby	Weryfikuje biznesowy wymóg loterii: każdy zestaw zawiera 6 unikalnych liczb.	Każdy wylosowany zestaw zawiera dokładnie 6 niepowtarzających się wartości. Test zdany.
3	poprawny	AkceptacyjnyPodsumowanieWystapienZawieraWszystkieLiczbyOd1Do49	Sprawdza, czy po wyświetleniu zestawów użytkownik otrzymuje pełne zestawienie statystyk dla wszystkich 49 liczb z zakresu loterii.	Wynik zawiera kompletne statystyki od Wystąpienia liczby 1: do Wystąpienia liczby 49:. Test zdany.
4	niepoprawny	AkceptacyjnyUzytkownikWpisujeBledneDane_OtrzymujeCzytelnyKomunikat	Symuluje sytuację, w której użytkownik wpisuje niepoprawne dane (np. tekst "zero" lub liczbę 0).	Aplikacja wyświetla zrozumiały komunikat o błędzie i nie generuje błędnych losowań. Test zdany.



LoteriaTests — Loteria.Acceptar

```
using System.Text.RegularExpressions;

namespace Loteria
{
    Odwołania: 0
    public class AcceptanceTests
    {
        // Helper symulujący interakcję użytkownika z aplikacją
        Odwołania: 4 | 4/4 passing
        private string UruchomAplikacjeJakoUzytkownik(string wpisanaWartosc)
        {
            using (StringReader sr = new StringReader(wpisanaWartosc + Environment.NewLine))
            using (StringWriter sw = new StringWriter())
            {
                Console.SetIn(sr);
                Console.SetOut(sw);

                Program.Main(new string[0]);

                return sw.ToString();
            }
        }

        // TEST 1 (poprawny): Scenariusz - zamawianie określonej liczby losowań
        [Fact]
        Odwołania: 0
        public void Akceptacyjny_UzytkownikGenerujeZestawy_OtrzymujePrawidlowaLiczbeLosowan()
        {
            // Użytkownik chce wygenerować 3 zestawy
            string wejscieUzytkownika = "3";

            // Gdy: Użytkownik uruchamia program i podaje liczbę zestawów
            string wynik = UruchomAplikacjeJakoUzytkownik(wejscieUzytkownika);

            // Wtedy: Program wyświetla komunikat powitalny oraz 3 zestawy losowań
            Assert.Contains("Ile wygenerować losowań?", wynik);
            Assert.Contains("Losowanie 1:", wynik);
            Assert.Contains("Losowanie 2:", wynik);
            Assert.Contains("Losowanie 3:", wynik);
            Assert.DoesNotContain("Losowanie 4:", wynik);
        }

        // TEST 2 (poprawny): Scenariusz - weryfikacja zasad gry (6 unikalnych liczb)
        [Fact]
        Odwołania: 0
        public void Akceptacyjny_ZestawyLiczb_KazdyZestawRoznyIUnikalneLiczby()
        {
            // Given: Użytkownik zamawia 1 zestaw
            string wejscieUzytkownika = "1";

            // When: Program generuje losowanie
            string wynik = UruchomAplikacjeJakoUzytkownik(wejscieUzytkownika);

            // Then: Dopasowujemy konkretną linię "Losowanie 1: ..." aż do końca wiersza
            Match match = Regex.Match(wynik, @"Losowanie 1:\s*(.+)");
            Assert.True(match.Success, "Nie odnaleziono linii z wynikami losowania.");

            // Wyciągamy tylko cyfry z tej linii
            int[] liczby = Regex.Matches(match.Groups[1].Value, @"\d+")
                                .Cast<Match>()
                                .Select(m => int.Parse(m.Value))
                                .ToArray();

            Assert.Equal(6, liczby.Length); // Dokładnie 6 liczb
            Assert.Equal(6, liczby.Distinct().Count()); // Wszystkie liczby unikalne
            Assert.All(liczby, liczba => Assert.InRange(liczba, 1, 49)); // Zakres 1-49
        }
```

```
63          }
64
65          // TEST 3 (poprawny): Scenariusz – dostępność pełnego podsumowania statystyk
66          [Fact]
            Odwołania: 0
67          public void Akceptacyjny_PodsumowanieWystapien_ZawieraWszystkieLiczbyOd1Do49()
68          {
69              // Given: Użytkownik zamawia 2 zestawy
70              string wejscieUzytkownika = "2";
71
72              // When: Aplikacja kończy generowanie
73              string wynik = UruchomAplikacjeJakoUzytkownik(wejscieUzytkownika);
74
75              // Then: Wyświetlone są statystyki dla każdej z 49 liczb
76              for (int i = 1; i <= 49; i++)
77              {
78                  Assert.Contains($"Wystąpienia liczby {i}:", wynik);
79              }
80          }
81
82          // TEST 4 (niepoprawny): Scenariusz – obsługa błędnego wpisania liczby przez użytkownika
83          [Fact]
            Odwołania: 0
84          public void Akceptacyjny_UzytkownikWpisujeBledneDane_OtrzymujeCzytelnyKomunikat()
85          {
86              // Given: Użytkownik wpisuje błędne dane ("zero")
87              string wejscieUzytkownika = "zero";
88
89              // When: Użytkownik zatwierdza wejście
90              string wynik = UruchomAplikacjeJakoUzytkownik(wejscieUzytkownika);
91
92              // Then: Program wypisuje komunikat o błędzie i nie wyświetla sekcji wyników
93              Assert.Contains("Niepoprawna wartość. Podaj liczbę całkowitą większą od 0", wynik);
94              Assert.DoesNotContain("Zestawy wylosowanych liczb:", wynik);
95          }
96      }
97  }
98
```

4.3. GIT

Screen z git log commitów wprowadzających przypadki testowe:

```
commit 55efd86a651fda87b1ba60a3af215de6fa9439f2 (HEAD -> master)
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 12:16:17 2026 +0200

    Utworzenie czwartego testu akceptacyjnego – sprawdzania przyja
znej obsługi błędów wejściowych wprowadzonych przez użytkownika

commit 066e3c1d725d52eb72059bb918a84d0a92884b4f
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 12:11:54 2026 +0200

    Utworzenie trzeciego testu akceptacyjnego – sprawdzania obecno
ści pełnego zestawienia statystyk dla wszystkich 49 liczb

commit 13fa0e91870b9d72e13094392d6cc01f8d141410
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 12:05:54 2026 +0200

    Utworzenie drugiego testu akceptacyjnego – unikalności i zakre
su liczb w zestawach z perspektywy zasad gry

commit f0d88645b1b4e9838dff790915d456b980bbef22
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Sat Oct 24 12:00:40 2026 +0200

    Utworzenie pierwszego testu akceptacyjnego – sprawdzania gener
owania określonej liczby losowań zgodnie z żądaniem użytkownika
```

Podsumowanie

Bibliografia